



CAT

A shared workspace where people and AI discuss a codebase together

metaphor.z.github.io/cat

What it is

CAT is a small chat workspace for our group. It looks like Slack—a sidebar of channels, a message list, a box to type in—and for ordinary conversation that is all it is. What it adds is that the AI assistants in the room are *participants*: they have names, they are addressed like anyone else, and their replies are ordinary messages that everyone can read, quote and argue with. Nothing happens in a private side conversation.

The difference that matters is context. A channel is bound to a GitHub repository, and the agents in that channel have actually read it—the file tree, the README, the code. Asking `@claude` about `windfield_dynamic.py` in the `#multimodel` channel is not the same as pasting a snippet into a chatbot that has never seen the project. It knows what else is in there and how the pieces connect.

Getting in

There are no passwords. You enter your email address and the site sends you a sign-in link; clicking it signs you in on that browser. Your address has to be added to the workspace first, so ask Paul before your first attempt—otherwise the sign-in will simply be refused. The link is single use, expires in an hour, and must be opened in the browser you intend to work in.

Channels

Each channel is normally one codebase. `#multimodel` is about the Multimodel wind and hurricane repository, and its header links straight to it on GitHub. `#general` is the exception: no repository is attached to it, so it is where technical discussion goes when it is not about one particular codebase. An agent there will follow the question rather than steering you back to a repo it cannot see. It is cleared out periodically, so treat it as conversation rather than a record.

The agents

Three are available—`@claude`, `@codex` and `@gemini`—and you summon one by naming it in a message. They read the recent conversation as well as the repository, so you can ask a follow-up without restating everything.

They cannot change anything. An agent has no way to edit files or push code; if you ask for a change, it will describe the change concretely and say that a person has to make it. The one exception is deliberate and narrow: on a channel whose repository has been explicitly opened for it, someone with permission can ask an agent to open a *pull request*. That produces a branch and a PR for review. It is never merged automatically, and nothing reaches the main branch without a human approving it.

Summoning an agent costs money, so that permission is granted individually rather than to everyone. Everyone can read everything and talk to anyone; ask Paul if you need to invoke agents.

cat

Do not include any private or proprietary information.

CHANNELS

multimodel

AGENTS

- @claude
- @codex
- @gemini

PEOPLE

MODELING AND SIMULATION

- Paul (you) @paul

Paul

can invoke agents

Size Large

Sign out

multimodel

Discussion of the Multimodel wind/hurricane codebase.

github.com/metaphorz/Multimodel

AI

Claude (AGENT) 3:46 PM

Short version, Paul: it's a hurricane wind→loss sensitivity/uncertainty study built to satisfy Florida Commission Standard S-3 / Form S-6, with a Leaflet browser viewer as the front end. Three moving parts:

- Offline Python pipeline (pipeline/)** — builds static artifacts that the viewer loads.
 - Domain: build_grid.py + build_florida_boundary.py + add_place_names.py give the south-Florida grid of points (the ROA grid from pp. 336–341).
 - Hazard: parametric wind field models — Holland, Willoughby, Powell — in windfield.py / windfield_grid.py, with windfield_dynamic.py (and ..._batch.py) for time-marching a moving storm rather than a snapshot, and windfield_ua.py for the uncertainty-analysis variants. Surface effects come from build_roughness.py and build_roughness_directional.py (roughness varies with inflow direction, so gust factors are azimuth-dependent).
 - Consequence: build_exposure.py / build_exposure_tax.py and build_vulnerability.py produce the exposure and damage-function side; fill_form_s6.py emits the regulatory form.
 - Precompute: precompute_live.py and read_inputs.py are the plumbing — read inputs/FormS6Input.xlsx, run the design, dump JSON for the web app. build_all.sh / build_regular.sh orchestrate.
- Design + metamodel layer** — this is the part closest to your discipline. make_lhs_design.py, make_constrained_design.py, make_test_design.py generate the input designs over the uncertain parameters (Vmax, RMW, B, translation speed, roughness, etc.). fit_metamodels.py fits emulators to the expensive simulator output so the viewer can do interactive response-surface and Sobol' sensitivity queries without rerunning the field. holdout_points.py / holdout_test.py validate the emulators out-of-sample. If you want to dig into the design/emulator choices specifically, that's the statistician's area.
- Browser viewer (web/)** — viewer.js is the shell, windfield.js renders the field, contour.js does isotachs, anim.js animates the dynamic storm, analysis.js drives the sensitivity plots (Sobol indices, EP curves, AAL), poi.js / popup.js handle point picking. index.html + style.css. Entry point is the repo-root index.html and the start/stop scripts.
- Validation** — HistoricalIke/ plus historical_domain_ike.py and plot_top_storms_ike.py compare simulated integrated kinetic energy against historical storms (Ike as the reference case). tests/auto/ is a large collection of standalone check scripts — t0_convergence.py, check_duration_*, check_sobol.py, check_steady_vs_dyn_claims.py — that read more like numerical-experiment notebooks than unit tests, which is probably where you'll want to start reading.

Where do you want to go deeper — the dynamic vs. steady field formulation, or the design/emulator loop?

anthropic/claude-opus-5 - 3942 in / 1210 out - asked by Paul

Message #multimodel

Address @claude, @codex, @gemini to bring an agent in.

Send

The workspace, mid-conversation. Channels and the agent roster sit on the left, people below them with their discipline. The line under the reply records which model answered, what it cost, and who asked. Shown in the dark theme, and captured before **#general** and the light/dark control were added.

It knows who is asking

We are an interdisciplinary group, and a question about the wind field means something different depending on who asks it. So each member carries a discipline—meteorology, statistics, actuarial, geospatial, visualization, engineering, modeling and simulation, computer science—and each channel maps those disciplines to the actual directories and files in that repository that belong to them.

The practical effect is that answers are pitched at you rather than at a generic reader, and that an agent will point you to the specific files in your area instead of the whole tree. It will also say when a question really belongs to someone else's discipline and name the person, which is usually more useful than a confident answer from the edge of what it knows. Tell Paul your discipline when you are added, so this works from your first message.

Ground rules

Stay on topic. The channel name tells you the topic. **#multimodel** is about the codebase on GitHub for the PT Multimodel Tool, and discussion there should be about that. **#general** is for general discussion, which may stray from any one codebase, but it is still for technical comments and questions—not a social channel.

Everyone sees every channel. There is no private channel and no per-channel membership. That is a deliberate choice for a group of this size—under twenty people who already work together—and it is worth knowing before you write something you would not want the whole group reading.

Do not paste anything private or proprietary. Messages go to a database and, when you summon an agent, to a model provider. Unpublished data, personal information, and anything under an agreement that restricts where it may go do not belong in the chat. The interface says so at the

top of the sidebar, and it means it.

Assume the AI can be wrong. It has read the repository, which makes it unusually well informed and not infallible. Treat an agent's answer the way you would treat a capable colleague's quick reply: worth having, worth checking before you rely on it. Every reply records which model produced it, so you can see what you are dealing with.

Where things are

Everything lives in the browser at metaphorz.github.io/cat—there is nothing to install, on any machine. Sign in once per browser and it remembers you.

Two things in the sidebar are yours to set: **light or dark**, from the button at the top, and **size**, which scales the whole interface rather than the type alone. Both are remembered in that browser, so set them once per machine.

Messages appear for everyone the moment they are sent, so a conversation works in real time if people are around, and reads perfectly well later if they are not. You can write Markdown, and it renders: code in fenced blocks, tables, and mathematics in L^AT_EX between dollar signs, which is worth using—the agents write the statistics back to you as notation rather than prose.